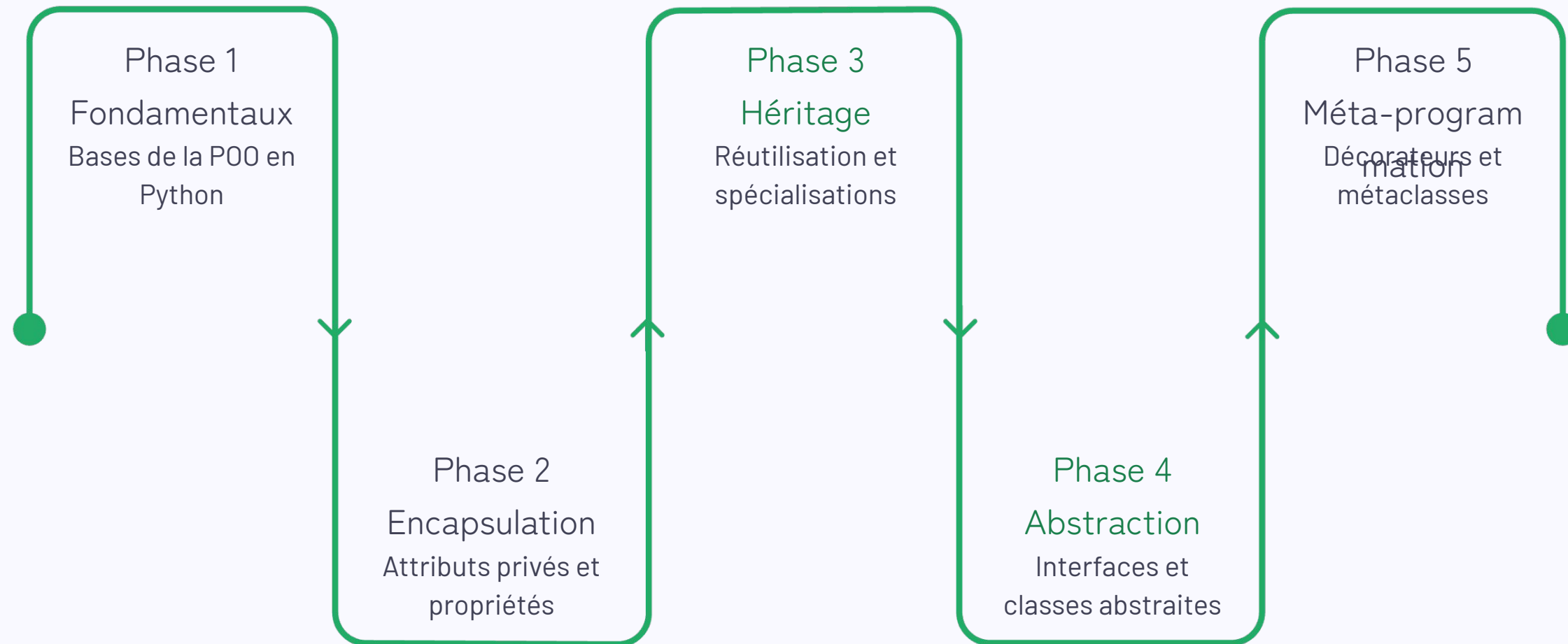


Maîtriser la Programmation Orientée Objet en Python

Un parcours complet en 6 phases pour passer des fondamentaux à l'architecture logicielle avancée – conçu pour les développeurs Python débutants à intermédiaires.



PHASE 1

Les Fondamentaux

Syntaxe de base et création d'objets en Python

Paradigme Objet

Comprendre la différence avec la programmation procédurale

Classes & Instances

`class` vs objet : déclarer et instancier

Constructeur & self

`__init__`, initialisation des états et liaison d'instance

Méthodes & Attributs

Comportements d'instance, portée des variables de classe

Paradigme Objet vs Procédural

Style Procédural

Le code est organisé autour de **fonctions** et de **données séparées**. On appelle des fonctions auxquelles on passe des données en paramètre.

```
def afficher(nom, race):  
    print(f"{nom} ({race})")  
  
afficher("Rex", "Labrador")
```

Style Objet

Le code regroupe **données et comportements** dans une même entité. L'objet connaît son propre état et sait quoi faire avec.

```
class Chien:  
    def __init__(self, nom, race):  
        self.nom, self.race = nom, race  
  
    def afficher(self):  
        print(f"{self.nom} ({self.race})")
```

 La POO favorise la modularité, la réutilisabilité et la lisibilité dans les projets de grande envergure.

Déclaration de Classe et Instanciation

Une **classe** est un modèle (blueprint). Un **objet** (instance) est une réalisation concrète de ce modèle.

```
class Voiture:  
    pass  
  
ma_voiture = Voiture()  
ta_voiture = Voiture()  
  
print(ma_voiture is ta_voiture)  # False
```

Classe

Le **plan** : définit la structure et les comportements communs à tous les objets du même type.

Instance

L'**objet créé** en mémoire : possède ses propres valeurs d'attributs, indépendantes des autres instances.

Le Constructeur `__init__`

La méthode `__init__` est appelée automatiquement lors de la création d'un objet. Elle initialise l'**état interne** de chaque instance.

```
class Voiture:
    def __init__(self, marque, vitesse):
        self.marque = marque
        self.vitesse = vitesse
```

```
v1 = Voiture("Renault", 180)
v2 = Voiture("Tesla", 250)
print(v1.marque, v2.marque)
```

📌 **Bonne pratique :** Initialisez tous les attributs dans `__init__` pour garantir un état cohérent dès la création.

L'Attribut `self` : Liaison d'Instance

`self` est une **référence à l'objet courant**. Il est automatiquement passé comme premier paramètre de toute méthode d'instance.

`self` permet à chaque objet d'accéder à **ses propres attributs**

Il n'est pas un mot-clé Python, mais une **convention universelle**

Sans `self`, les méthodes ne sauraient pas sur **quel objet** agir

```
class Compteur:
    def __init__(self):
        self.valeur = 0

    def incrementer(self):
        self.valeur += 1

c1, c2 = Compteur(), Compteur()
c1.incrementer()
c1.incrementer()
c2.incrementer()

print(c1.valeur, c2.valeur) # 2 1
```

Méthodes d'Instance

Les méthodes définissent les **comportements** d'un objet — ce qu'il sait faire. Elles opèrent sur les données propres à l'instance via `self`.

```
class Rectangle:
    def __init__(self, w, h):
        self.w, self.h = w, h

    def surface(self):
        return self.w * self.h

    def est_carre(self):
        return self.w == self.h
```

```
r = Rectangle(4, 4)
print(r.surface()) # 16
print(r.est_carre()) # True
```

Attributs de Classe vs d'Instance

Attributs de Classe

Partagés entre toutes les instances. Définis directement dans le corps de la classe.

```
class Animal:
    espece = "Mammifère"
    def __init__(self, nom):
        self.nom = nom
```

```
a = Animal("Rex")
print(a.espece) # Mammifère
Animal.espece = "Reptile"
print(a.espece) # Reptile
```

Différences clés

Classe

Partagé, défini hors `__init__`. Utile pour les constantes et compteurs globaux.

Instance

Propre à chaque objet, défini via `self` dans `__init__`.

PHASE 2

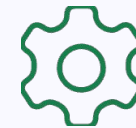
Encapsulation et Interface

Protéger les données internes et exposer des interfaces propres et contrôlées.



Visibilité

Conventions `_protected` et `__private` pour contrôler l'accès aux attributs.



@property

Getters et Setters élégants pour valider et contrôler les modifications d'état.



Dunder Methods

`__str__`, `__repr__`, `__del__` : personnaliser le comportement natif.



Surcharge d'opérateurs

`__add__`, `__eq__`, `__len__` : rendre vos objets pythoniques.

Visibilité et Convention de Nommage

Python n'a pas de vrai système de contrôle d'accès, mais des **conventions de nommage** signalent l'intention du développeur.

`attribut`

Public : accessible librement depuis l'extérieur de la classe. Usage par défaut.

`_attribut`

Protégé : usage interne ou sous-classes. La convention dit "ne pas toucher directement".

`__attribut`

Privé : Python applique le *name mangling* (`_Classe__attribut`) pour décourager l'accès externe.

```
class CompteBancaire:
    def __init__(self, solde):
        self.__solde = solde

    def deposer(self, montant):
        if montant > 0:
            self.__solde += montant
```

Le Décorateur @property

Exposer un attribut comme une propriété calculée tout en gardant le contrôle sur sa lecture et son écriture.

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, v):
        if v < -273.15: raise ValueError()
        self._celsius = v

    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32
```

Avantages de @property

- Validation
Interceptez et validez les valeurs avant qu'elles soient assignées.
- Calcul à la volée
Retournez des valeurs calculées sans stocker de données redondantes.
- Rétrocompatibilité
Transformez un attribut public en propriété sans changer l'API.

Méthodes Dunder (Magic Methods)

Les méthodes **dunder** (double underscore) permettent à vos objets de s'intégrer naturellement dans le langage Python.

```
class Livre:
    def __init__(self, t, a, p):
        self.t, self.a, self.p = t, a, p
    def __str__(self):
        return f"'{self.t}' par {self.a}"
    def __repr__(self):
        return f"Livre({self.t!r}, {self.a!r}, {self.p})"
    def __del__(self):
        print(f"Suppression : {self.t}")

livre = Livre("1984", "Orwell", 328)
print(livre)      # '1984' par Orwell
print(repr(livre)) # Livre('1984', 'Orwell', 328)
```

Surcharge d'Opérateurs

Définir comment vos objets réagissent aux opérateurs Python standards (+, ==, len()...).

```
class Vec:
    def __add__(self, o):
        return Vec(self.x + o.x)
```

```
v1 + v2 # appelle __add__
```

Méthodes courantes

`__add__`

Opérateur +

`__eq__`

Opérateur ==

`__len__`

Fonction len()

`__getitem__`

Accès par index `obj[i]`

PHASE 3

Hiérarchie et Réutilisation

Structurer le code grâce à l'héritage pour éviter la duplication et favoriser l'extensibilité.

Héritage Simple

Une classe **enfant** hérite automatiquement de tous les attributs et méthodes de la classe **parent**, et peut en ajouter de nouveaux.

```
class Animal:
    def respirer(self): print("Respire...")

class Chien(Animal):
    def aboyer(self): print("Woof!")

rex = Chien()
rex.respirer()
rex.aboyer()
```

📌 Règle : utilisez l'héritage uniquement quand la relation **"EST-UN"** est vraie. Un Chien est *un* Animal ✓

La Fonction `super()`

`super()` donne accès à la classe parente, permettant d'**étendre** son comportement plutôt que de le remplacer entièrement.

```
class Employe:
    def __init__(self, nom, salaire):
        self.nom, self.salaire = nom, salaire

class Manager(Employe):
    def __init__(self, nom, salaire, equipe):
        super().__init__(nom, salaire)
        self.equipe = equipe

    def presenter(self):
        super().presenter()

        print(f"Manage {len(self.equipe)} pers.")
```

Pourquoi utiliser `super()` ?

1 Évite la duplication

On n'écrit pas deux fois la logique déjà présente dans le parent.

2 Respecte la hiérarchie MRO

Indispensable avec l'héritage multiple pour suivre l'ordre de résolution.

3 Facilite la maintenance

Modifier le parent se propage automatiquement aux enfants.

Redéfinition de Méthodes (Overriding)

Une classe enfant peut **redéfinir** une méthode héritée pour adapter son comportement à sa propre nature.

```
class Forme:  
    def surface(self): raise NotImplementedError
```

```
class Cercle(Forme):  
    def __init__(self, r): self.r = r  
    def surface(self): return 3.14 * self.r**2
```

```
class Carre(Forme):  
    def __init__(self, c): self.c = c  
    def surface(self): return self.c**2
```

```
print(Cercle(5).surface()) # 78.5
```

```
print(Carre(4).surface()) # 16.0
```

Héritage Multiple et MRO

Python permet l'héritage depuis **plusieurs classes**. Le **MRO** (Method Resolution Order) définit l'ordre dans lequel Python cherche une méthode.

```
class A:
    def saluer(self): print("A")
class B(A):
    def saluer(self): print("B")
class C(A):
    def saluer(self): print("C")
class D(B, C): pass

d = D()
d.saluer()    # B

print(D.__mro__)
```

Règle de gauche à droite

Les parents listés en premier ont la priorité.

L'algorithme C3

Python utilise l'algorithme **C3 linéarisation** pour calculer le MRO, garantissant un ordre cohérent et prévisible.

❏ Utilisez `Classe.__mro__` ou `help(Classe)` pour inspecter l'ordre de résolution.

Profondeur avant largeur

On remonte toute une branche avant de passer à la suivante.

Composition vs Héritage

Deux stratégies fondamentales pour réutiliser du code — choisissez en fonction de la **nature de la relation**.

Héritage : relation "EST-UN"

Un **Chien** EST-UN **Animal**

```
class Animal:
    def respirer(self): pass

class Chien(Animal):
    def aboyer(self): pass
```

Quand ? Quand la classe enfant est véritablement un cas particulier du parent.

Composition : relation "A-UN"

Une **Voiture** A-UN **Moteur**

```
class Moteur:
    def demarrer(self): pass

class Voiture:
    def __init__(self):
        self.moteur = Moteur()

    def demarrer(self):
        self.moteur.demarrer()
```

Quand ? Quand un objet *contient* un autre sans en être un sous-type.

  Préférez la **composition** par défaut — elle crée un couplage plus faible et un code plus flexible.

Abstraction et Architecture

Définir des contrats clairs, exploiter le polymorphisme et organiser intelligemment vos classes.



Classes Abstraites

Module `abc` et `@abstractmethod` pour imposer une interface à implémenter.



Polymorphisme

Appeler la même méthode sur des types différents et obtenir des comportements adaptés.



Duck Typing

La philosophie dynamique de Python : "Si ça canane comme un canard, c'est un canard."



@classmethod & @staticmethod

Des méthodes qui opèrent sur la classe elle-même, pas sur une instance particulière.

Classes Abstraites avec abc

Une classe abstraite définit un **contrat** : toute sous-classe doit implémenter les méthodes abstraites déclarées.

```
from abc import ABC, abstractmethod

class Forme(ABC):
    @abstractmethod
    def surface(self): pass

class Triangle(Forme):
    def __init__(self, b, h): self.b, self.h = b, h
    def surface(self): return 0.5 * self.b * self.h
```

Interfaces et Duck Typing

En Python, pas besoin d'interfaces formelles. Si un objet possède les bonnes méthodes, il **joue le rôle** attendu.

```
class Chien:
    def parler(self): return "Woof!"
class Robot:
    def parler(self): return "Bip bop!"

def faire_parler(obj):
    print(obj.parler())

for entite in [Chien(), Robot()]:
    faire_parler(entite)
```

La philosophie pythonique

"Si ça marche comme un canard et cancanne comme un canard, alors c'est un canard."

Python vérifie le **comportement**, pas le type. Cela rend le code plus flexible et plus facile à étendre.

❏ Utilisez `hasattr()` ou `isinstance()` uniquement quand vous avez besoin d'une vérification explicite de type.

Polymorphisme

Le polymorphisme permet d'**utiliser des objets de types différents** de façon uniforme, en appelant la même méthode sur chacun d'eux.

```
class Paiement:
    def payer(self, montant): pass

class Carte(Paiement):
    def payer(self, m): print(f"Carte: {m}€")

class PayPal(Paiement):
    def payer(self, m): print(f"PayPal: {m}€")

for m in [Carte(), PayPal()]:
    m.payer(100)
```

@classmethod et @staticmethod

@classmethod

Reçoit la **classe** (`cls`) comme premier argument. Utile pour les *factory methods*.

```
class Date:
    @classmethod
    def depuis_chaine(cls, s: str):
        j, m, a = map(int, s.split('/'))
        return cls(j, m, a)
```

```
d = Date.depuis_chaine("25/12/2024")
```

@staticmethod

Ne reçoit ni `self` ni `cls`. Simple fonction utilitaire liée à la classe.

```
class MathUtils:
    @staticmethod
    def est_premier(n: int) -> bool:
        if n < 2: return False
        for i in range(2, int(n**0.5) + 1):
            if n % i == 0: return False
        return True
```

```
print(MathUtils.est_premier(17))
```


PHASE 5



Concepts Avancés & Méta-programmation

Aller au-delà des patterns classiques pour écrire un code Python moderne, puissant et élégant.

Décorateurs

Modifier le comportement de fonctions, méthodes et classes sans les toucher.

@dataclass

Génération automatique de code boilerplate pour les classes de données.

Exceptions

Créer des hiérarchies d'exceptions métier expressives et précises.

Métaclasses

Contrôler la création de classes elle-même avec `type`.

`__slots__`

Optimiser la mémoire en limitant les attributs autorisés sur une instance.

Décorateurs de Fonctions et Méthodes

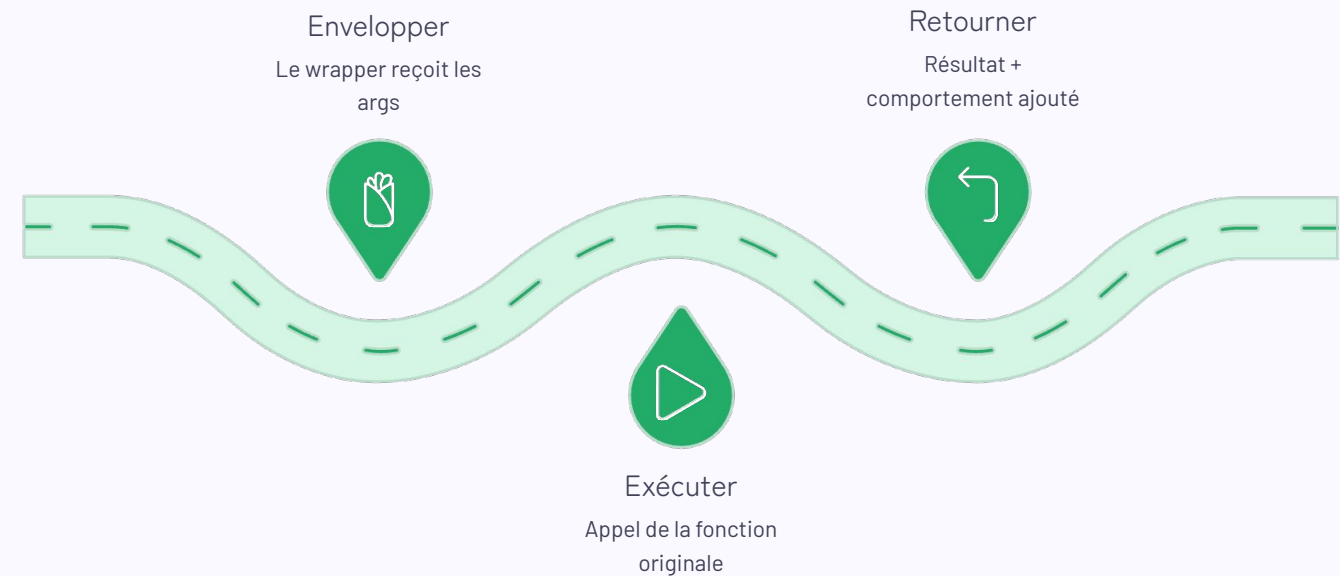
Un décorateur est une **fonction qui enveloppe** une autre fonction pour modifier ou enrichir son comportement sans la modifier.

```
from functools import wraps

def mon_decorateur(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("Avant")
        res = func(*args, **kwargs)
        print("Après")
        return res
    return wrapper

@mon_decorateur
def saluer(nom):
    print(f"Bonjour {nom}")
```

Pattern classique



Décorateurs de Classes

Les décorateurs peuvent aussi s'appliquer à des **classes entières** pour les modifier ou les enregistrer au moment de leur définition.

```
def ajouter_methode_info(cls):  
    def info(self):  
        print(f"[{cls.__name__}] {self.__dict__}")  
    cls.info = info  
    return cls
```

```
@ajouter_methode_info
```

```
class Produit:  
    def __init__(self, nom, prix):  
        self.nom = nom; self.prix = prix
```

```
p = Produit("Stylo", 1.5)
```

```
p.info()
```

Classes de Données avec @dataclass

Le décorateur `@dataclass` génère automatiquement `__init__`, `__repr__`, et `__eq__` en fonction des annotations de type.

```
from dataclasses import dataclass

@dataclass
class Etudiant:
    nom: str
    age: int
    notes: list = None

e = Etudiant("Alice", 22, [15, 17])
print(e)
# Etudiant(nom='Alice', age=22, notes=[15, 17])
print(e == Etudiant("Alice", 22, [15, 17]))
# True
```

Ce que @dataclass génère

`__init__`

Constructeur basé sur les annotations de type.

`__repr__`

Représentation lisible de l'objet.

`__eq__`

Comparaison basée sur les valeurs des champs.



Options utiles : `frozen=True` pour un objet immuable, `order=True` pour activer `<`, `>`.

Exceptions Personnalisées

Créer des **hiérarchies d'exceptions métier** rend le code plus expressif et les erreurs plus faciles à gérer.

```
class AppError(Exception): pass

class ValidationError(AppError):
    def __init__(self, field, msg):
        self.field = field
        super().__init__(f"{field}: {msg}")

def validate_age(age):
    if not (0 <= age <= 150):
        raise ValidationError("age", "invalid")

try:
    validate_age(-5)
except ValidationError as e:
    print(e)
```

PHASE 6



Design et Qualité Logicielle

Les principes et outils qui distinguent un code qui "fonctionne" d'un code **maintenable, extensible et professionnel**.



Principes SOLID

5 règles fondamentales pour concevoir des classes robustes et évolutives.



Design Patterns

Singleton, Factory, Strategy, Observer : solutions éprouvées à des problèmes récurrents.



UML de Base

Schématiser ses classes et leurs relations avant de coder pour mieux architecturer.

Les Principes SOLID

1

S — Single Responsibility

Une classe = une seule raison de changer. Séparer les préoccupations.

2

O — Open/Closed

Ouverte à l'extension, fermée à la modification. Étendre sans casser.

3

L — Liskov Substitution

Une sous-classe doit être substituable à sa classe parent sans surprises.

4

I — Interface Segregation

Des interfaces petites et spécifiques plutôt qu'une grande interface fourre-tout.

5

D — Dependency Inversion

Dépendre d'abstractions, pas de concrétisations.

SOLID en Pratique : SRP et OCP

❌ Violation du SRP

```
class Rapport:
    def calculer(self): ...
    def formater_html(self): ...
    def sauvegarder(self): ...
```

✅ SRP respecté

```
class Rapport:
    def calculer(self): ...

class FormateurHTML:
    def formater(self, rapport): ...

class Sauvegarde:
    def sauvegarder(self, contenu): ...
```

✅ OCP : Extensible

```
class Remise(ABC):
    @abstractmethod
    def calculer(self, prix): ...

class RemiseStandard(Remise):
    def calculer(self, prix): return prix * 0.9

class RemisePremium(Remise):
    def calculer(self, prix): return prix * 0.7
```


Design Pattern : Singleton

Le **Singleton** garantit qu'une classe n'a **qu'une seule instance** et fournit un point d'accès global. Utile pour les configurations, connexions DB, loggers.

```
class Singleton:
    _instance = None
    def __new__(cls):
        if not cls._instance:
            cls._instance = super().__new__(cls)
        return cls._instance

s1, s2 = Singleton(), Singleton()
print(s1 is s2) # True
```

Cas d'usage courants

- Configuration applicative
Un seul objet config partagé dans tout le projet.
- Pool de connexions
Réutiliser une connexion base de données coûteuse.
- Système de logs
Un logger global cohérent dans toute l'application.

Design Pattern : Factory

Le pattern **Factory** délègue la **création d'objets** à une méthode dédiée, découplant le code client du type concret instancié.

```
class Notification(ABC):
    @abstractmethod
    def envoyer(self, msg): ...

class Email(Notification):
    def envoyer(self, msg): print(f"Email: {msg}")

class Factory:
    _map = {"email": Email, "sms": SMS}
    @classmethod
    def creer(cls, type): return cls._map[type]()

notif = Factory.creer("sms")
notif.envoyer("Hello!")
```

Design Pattern : Strategy

Le pattern **Strategy** définit une famille d'algorithmes interchangeables. On peut changer la stratégie **à l'exécution** sans modifier le contexte.

```
from abc import ABC, abstractmethod

class TriStrategy(ABC):
    @abstractmethod
    def trier(self, data: list): ...

class TriRapide(TriStrategy):
    def trier(self, data): return sorted(data)

class Trieur:
    def __init__(self, s): self._s = s
    def executer(self, data):
        return self._s.trier(data)

t = Trieur(TriRapide())
print(t.executer([3,1,4,1,5]))
```

Pourquoi Strategy ?

Ouvert/Fermé

Ajouter un algorithme = créer une nouvelle classe, sans toucher au contexte.

Testabilité

Chaque stratégie est testée isolément.

Flexibilité runtime

Changer l'algorithme dynamiquement selon le contexte (taille des données, perf...).

Design Pattern : Observer

Le pattern **Observer** établit une relation *un-à-plusieurs* : quand un objet change d'état, tous ses **observateurs** sont notifiés automatiquement.

```
class Observateur(ABC):
    @abstractmethod
    def update(self, msg): ...

class Sujet:
    def __init__(self): self._obs = []
    def abonner(self, o): self._obs.append(o)
    def notifier(self, m):
        for o in self._obs: o.update(m)

class Boutique(Sujet):
    def commande(self, c): self.notifier(f"Cmd: {c}")

class Stock(Observateur):
    def update(self, m): print(f"Stock: {m}")
```

UML de Base : Schématiser ses Classes

L'**UML** (Unified Modeling Language) permet de visualiser la structure d'un système avant de coder. Maîtriser le **diagramme de classes** est essentiel.

Notations fondamentales



Association

Relation simple entre deux classes.



Héritage

Flèche creuse : "EST-UN".



Composition

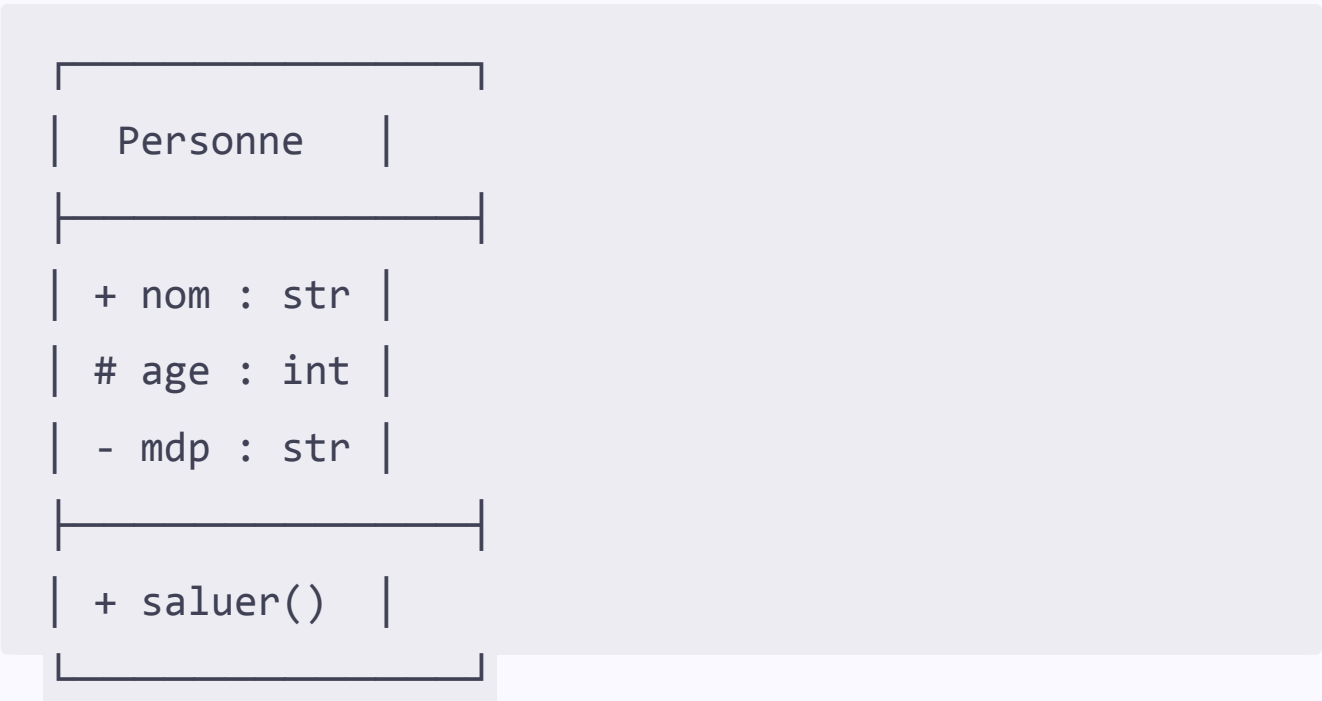
Losange plein : dépendance forte "A-UN".



Réalisation

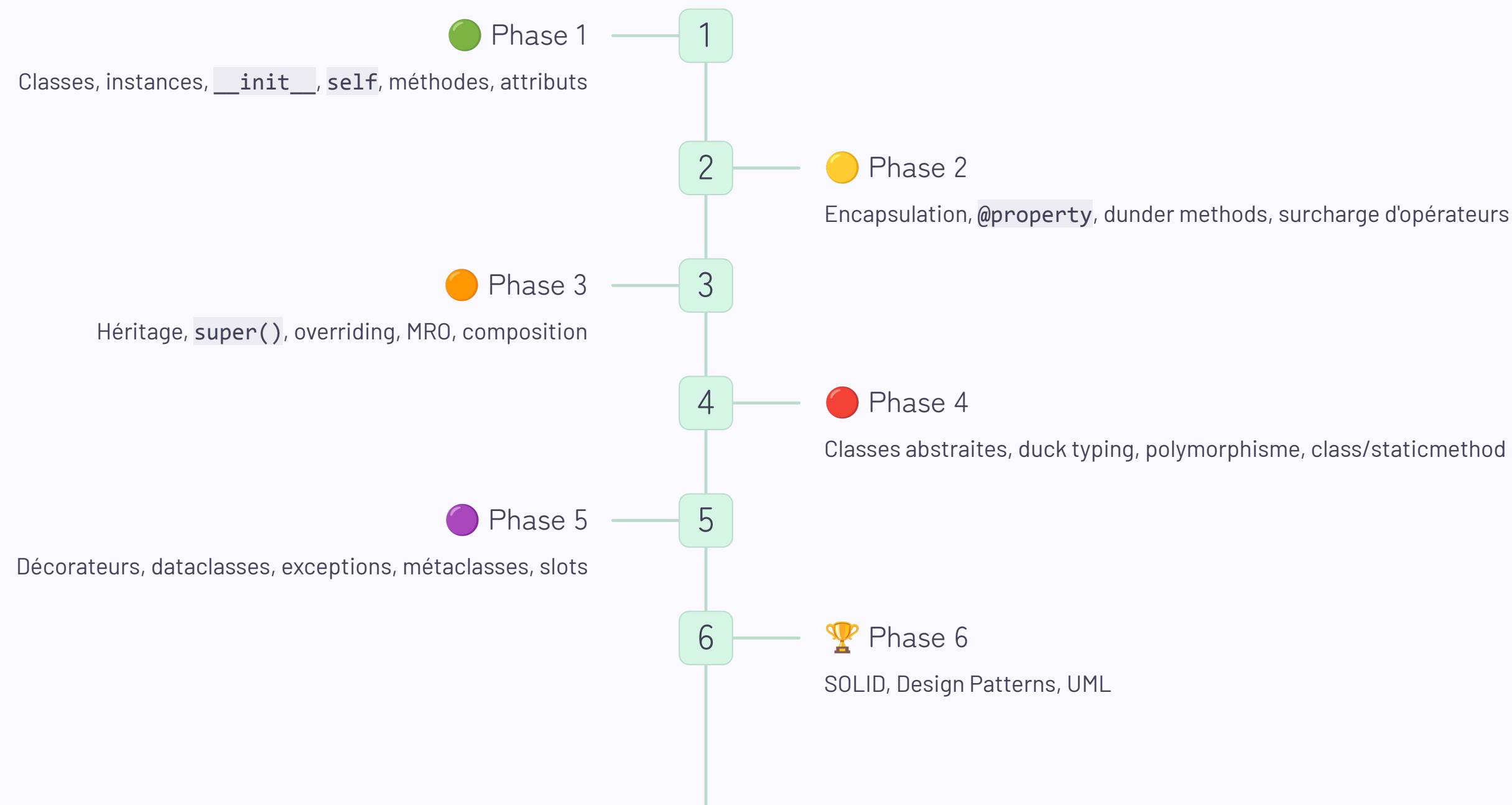
Flèche pointillée : implémentation d'interface.

Visibilité dans une classe UML



Concevoir votre diagramme UML **avant de coder** réduit les refactorisations coûteuses et clarifie l'architecture.

Récapitulatif du Parcours



Les 4 Piliers de la POO

Ces quatre piliers travaillent ensemble pour créer un code **modulaire, maintenable et extensible**. Chaque phase du cours contribue à en maîtriser un ou plusieurs.

Prochaines Étapes

Vous avez maintenant une feuille de route complète. Voici comment progresser efficacement :



Pratiquer chaque concept

Codez un mini-projet par phase. Un `Zoo`, une `Banque`, un `Système de commandes`...



Écrire des tests unitaires

Utilisez `unittest` ou `pytest` pour tester chaque classe — cela force une bonne conception.



Relire du bon code Python

Explorez les sources de `pathlib`, `dataclasses`, ou Django — la P00 en production.



Appliquer SOLID sur vos projets

Refactorisez un ancien projet en appliquant les principes vus en Phase 6.

Merci & Bonne pratique !

Ce parcours vous donne toutes les clés pour écrire du Python orienté objet **professionnel, lisible et maintenable**. La maîtrise vient avec la pratique — codez, expérimentez, itérez.

📖 **Ressource recommandée** : La documentation officielle Python (docs.python.org) et le livre "Fluent Python" de Luciano Ramalho sont d'excellents compagnons pour approfondir chaque concept de ce cours.